

slam_toolbox の Message Filter dropping message ... queue is full 調査レポート

- 日付: 2026-05-26
- 環境: Docker コンテナ rerobot_env (ROS 2 Jazzy + ros2_canopen + slam_toolbox)
- 対象ブランチ: main (HEAD: e602f54 feat: add slam_toolbox configuration and launch files)
- 関連ファイル:
 - src/rerobot_bringup/launch/rerobot_bringup.launch.py
 - src/rerobot_bringup/launch/slam.launch.py
 - src/rerobot_bringup/config/slam_toolbox.yaml
 - src/epos4_controller/src/epos4_odometry.cpp

TL;DR

slam_toolbox の scan_queue_size がライブラリ既定値の 1 のまま運用されており、/scan (40 Hz) と odom → base_link TF (20 Hz) のレート差で各 scan ごとに発生する TF 解決待ちが、即座にキューを溢れさせて tf2_ros::MessageFilter が古い scan を drop していた。slam_toolbox.yaml に scan_queue_size: 10 (+ transform_timeout: 0.5) を追記して解消。

症状

rerobot_bringup.launch.py (bringup) 単体では何の警告も出ず正常起動するが、slam.launch.py を同時に起動した瞬間から async_slam_toolbox_node が次のログを 2.5 秒間隔で吐き続け、map が生成されない。

```
[async_slam_toolbox_node-1] [INFO] [...] [slam_toolbox]: Message Filter dropping message:
  frame 'laser' at time 1779798256.067 for reason 'discarding message because the queue is full'
```

bringup 単体でエラーが出ないのは「正常」ではなく、このログを吐く主体 (slam_toolbox) がいないだけで、根本の TF/topic 構成は最初から slam を通すには不十分だった。

切り分けの記録

1. プロセス・トピック・TF の生存確認

bringup 起動後、ノード/トピックは期待どおり揃っていた。

項目	観測値	備考
/scan rate	39.96 Hz (25 ms 周期, 標準偏差 0.17 ms)	HOKUYO UTM-30LX (urg_node_driver)

項目	観測値	備考
/odom rate	20.00 Hz (50 ms 周期)	epos4_odometry、PDO sync 50 ms に追従
odom → base_link TF	正常 publish (20 Hz)	epos4_odometry が broadcast
base_link → laser TF	/tf_static で z=0.714 固定	robot_state_publisher + URDF laser_joint

→ TF chain (odom → base_link → laser) は完全に成立しており、TF 自体は失われていない。

2. stamp の時刻系比較

roslpy で /scan と /odom をそれぞれ 2 秒間サブスクライブし、各メッセージの header.stamp と受信 wall-clock の差 (age) を測定した。

```
avg /scan age = -3.29 ms   ← scan stamp は wall-clock より 3.3 ms 未来寄り
avg /odom age =  6.06 ms   ← odom stamp は wall-clock より 6 ms 過去
                             (stamp 自体は publish 0.6〜0.9 ms 前のものではほぼ即時)
```

urg_node は scan stamp に「次サンプル予測時刻」を入れるため僅かに未来側にずれ、epos4_odometry は CANopen の PDO 受信時刻 (現在より僅かに過去) を入れる。両者のクロックは同じ system clock で、系統的な数百 ms 級ズレは無い。

3. slam_toolbox の実行時パラメータ確認

```
$ ros2 param dump /slam_toolbox | grep -Ei
"queue|timeout|throttle|transform"
  scan_queue_size: 1           ← ライブラリ既定値
  throttle_scans: 1
  transform_publish_period: 0.02
  transform_timeout: 0.2
```

slam_toolbox.yaml で scan_queue_size を指定していなかったため、slam_toolbox ライブラリ側の既定値 1 がそのままセットされていた。

根本原因

tf2_ros::MessageFilter は /scan を購読する際、各 scan の header.stamp 時刻で odom_frame → scan.header.frame_id の TF を lookup する。本機の構成では:

```
scan 周期      : 25 ms (40 Hz)
odom TF 周期    : 50 ms (20 Hz)
```

```
scan stamp      : wall-clock の +3.3 ms 未来寄り
TF buffer 最新 : wall-clock の -0.6 ms 過去
```

scan を受信した瞬間、TF buffer の最新 stamp は **scan stamp より ~4 ms 過去** にしか存在しない。tf2 は extrapolation を許可しないので、次の **odom → base_link** TF (最大 50 ms 後) を待つ必要がある。

- 最悪ケース: scan を受信してから次の TF 到着まで ~50 ms 待機
- その間に次の scan が 25 ms 後に到着
- **scan_queue_size: 1** ではキューに 1 件しか保持できず、次の scan の到着で即 overflow
- → 古い scan は **discarding message because the queue is full** で drop

そのため /scan のうち相当割合 (ほぼ全部) が **slam_toolbox** 内部のマッチング段に到達できず、map が生成されなかった。

drop ログ自体は **tf2_ros** 側で約 2.5 秒に 1 行に rate-limit されるので、ログ密度は低く見えるが、実際の drop は scan rate と同程度で発生していた。

修正

src/rerobot_bringup/config/slam_toolbox.yaml に 2 行追加。rerobot_bringup は **--symlink-install** でビルドされているため、yaml 編集のみで install 側にも即時反映される (再ビルド不要)。

```
slam_toolbox:
  ros__parameters:
    ...
    # tf2_ros::MessageFilter のキュー長。slam_toolbox の素のデフォルトは 1 で、
    # /scan 40Hz と /odom 20Hz (= odom→base_link TF が 50ms 周期) の組み合わせ
    # 各 scan で「TF buffer 最新 < scan.stamp」が常に起き、次の odom TF を待つ間
    # すぐキューが overflow して "queue is full" で全 scan が drop される。
    scan_queue_size: 10
    # scan stamp と TF stamp の余裕。0.2s でも理屈上は足りるが、起動直後の TF
    # ウォームアップやレート揺らぎを吸うため 0.5s に拡張。
    transform_timeout: 0.5
```

なぜこの値か

- **scan_queue_size: 10**
 - 最悪ケースの待ち時間 50 ms に対し、scan 1 個あたり 25 ms。 $\text{ceil}(50/25) = 2$ 件を保持できればキューは溢れない。10 にしておけば /odom が瞬間的にスタックしても $10 \times 25 \text{ ms} = 250 \text{ ms}$ 分の余裕を持って吸収できる。
- **transform_timeout: 0.5**
 - odom → base_link の最大ギャップ (50 ms) + 起動直後の TF buffer ウォームアップを考慮しても余裕。これより小さくしても通常運用上は問題ないが、起動直後の初回 scan を取りこぼさないためのマージン。

検証

項目	修正前	修正後
<code>queue is full</code> ログ件数 (slam 起動後 ~30 秒)	12+ 件	0 件
<code>slam_toolbox</code> Lifecycle 遷移	configure → activate OK	configure → activate OK
<code>map</code> → <code>odom</code> TF broadcast	出していない	正常 broadcast (identity 行列)
<code>/map</code> topic	未 publish	5 秒周期で publish (<code>map_update_interval: 5.0</code> 通り)

検証手順 (bringup を維持したまま slam だけ再起動):

```
docker exec rerobot_env bash -lc 'pkill -f slam.launch.py; pkill -f
async_slam_toolbox'
docker exec -d rerobot_env bash -lc \
  'source /opt/ros/jazzy/setup.bash && source /workspace/install/setup.bash
&& \
  ros2 launch rerobot_bringup slam.launch.py > /tmp/slam.log 2>&1'
sleep 15
grep -c "queue is full" /tmp/slam.log      # → 0
ros2 param get /slam_toolbox scan_queue_size # → Integer value is: 10
ros2 param get /slam_toolbox transform_timeout # → Double value is: 0.5
ros2 run tf2_ros tf2_echo map odom -r 1    # → identity 行列が継続的に取れ
る
```

教訓 / 今後の予防

- 1. `scan_queue_size` は `slam_toolbox` を採用する全ロボットで明示する。ライブラリ既定の `1` は LiDAR rate ≤ TF rate の構成でしか成立しない設計で、本機のように `/scan` が `/odom`-TF より高レートな構成では必ず破綻する。
- 2. 「上位ノードが居ないからエラーが出ない」を正常と勘違いしない。bringup 単体で `tf2_ros::MessageFilter` の警告が出なかったのは、購読する側 (slam_toolbox) がいなかっただけで、TF/topic 系の整合性とは無関係だった。今後は slam を立ち上げる前提で TF rate と scan rate のレート差を意識する。
- 3. `/odom rate` を上げる選択肢もある (PDO 周期を 50 ms → 25 ms 等)。ただし CANopen バス負荷と引き換えになるので、SLAM 用に大きくする必要が無ければ `scan_queue_size` 側で吸収するほうが安価で安全。
- 4. `epos4_odometry` の `Synchronizer (50 ms maxIntervalDuration)` が間欠的に pair を取り逃すと `/odom` が落ちる時間帯が生まれ、TF buffer のラグが一時的に大きくなる。今回設定した `transform_timeout: 0.5` の余裕はこの保険も兼ねている。将来 PDO 周期や同期方針を変える場合は、両パラメータを併せて再評価 すること。